

第四章 第四节 Python面向对象编程实训

在学习 Python 面向对象编程技术后，以按揭贷款为案例，利用 Python 面向对象技术，对其进行建模，实践面向对象分析与编程。

- **第1关按揭贷款——定义抽象类**

任务描述

本关主题是利用 Python 面向对象编程技术，对按揭贷款的问题，进行面向对象建模与编程。请仔细阅读下面“相关知识”中的内容，理解每个函数所需完成的操作，补全相关函数，以实现计算按揭贷款的要求。

第四章 第四节 Python面向对象编程实训

• 第1关按揭贷款——定义抽象类

相关知识

2008 年美国的次贷危机导致全球经济下滑，至今还未完全恢复。次贷危机的一个因素是很多房主都采用按揭贷款（Mortgage）来购房，而未考虑按揭贷款带来的不可预知的后果。

早期的按揭贷款非常简单，房主从银行借钱，然后在接下来的 15~30 年间，每月向银行支付一个固定额度还钱。贷款到期后，银行收回借款和一定的利息，而房主完全地拥有了住房。

20 世纪末期，按揭贷款变得复杂了，人们可以选择在按揭开始时，支付“几个点”以降低贷款利率，1 个点指的是贷款额的 1%；或者选择多利率贷款，通常初始利息率非常低（称为引诱利率），然后慢慢攀升，因此每个月还给银行的钱是不固定的。尽管提供多种贷款方式供选择是好事，但是无良的放贷商通常不会详细解释各种贷款方式的长远影响，这往往会给贷款者将来的生活造成重大的影响。

为了了解按揭贷款的规则，我们可以将这些规则变成计算机能理解的程序语句，然后利用计算机问题求解，来帮助人们选择合适的贷款方式。下面考虑三种贷款方式：

- 固定利息率，不支付任何点；
- 固定利息率，支付一定的点；
- 可变利息率，低引诱利率，随后高利息率。

第四章 第四节 Python面向对象编程实训

在学习 Python 面向对象编程技术后，以按揭贷款为案例，利用 Python 面向对象技术，对其进行建模，实践面向对象分析与编程。

本关中，我们利用面向对象的程序设计技术，来研究按揭贷款：各种贷款选择除了有共同点（每次贷款包含本金和利息）外，又有不同点（如利率、是否支付点等），因此可将按揭贷款中的贷款，设计成一个类 Mortgage，而 3 种贷款设计成它的子类。由于贷款类并不作具体的，诸如计算利息等这类计算，故可以将其设计成抽象类（abstract class），只包含必要的供子类使用的函数，而不做具体的实现，也不建议直接实例化这个类。

接下来我们对 Mortgage 类和函数 findPayment 进行说明：

•每月应支付给银行的还款由函数 findPayment 计算，公式为：

$$loan * (1+r)^m - 1r * (1+r)^m$$

其中：loan 为贷款额，r 是月利息率，m 是贷款月数。

•Mortgage 对象初始化时，需要指定贷款额（loan）、月利息率（rate）、贷款期限（months，以月计），以及每月开始时—即到该月—已经支付的金额（paid，是一个列表）、每月开始时还剩的贷款额（owed，是一个列表）、每月应付金额（payment），以及对贷款类型的描述（legend）。__init__ 函数中的参数 loan 为贷款额，months 为还款月数，annRate 为年利息。例如年利息为 6.5%，则初始化时，annRate 的值为 6.5；

•makePayment 方法用于记录贷款的支付情况，每月支付给银行的金额，一部分用于支付利息，一部分用于归还本金。首先，在 paid 中，添加当月还款金额；然后，用前一月结束时，还欠银行的钱减去上一个月的利息，即为本月已还的本金。则本月还欠银行的本金，为上月本金减去本月已还本金。因此，该方法中，同时对 paid 和 owed 属性值进行了修改；

•getTotalPaid 方法用于计算支付给银行的本金和利息总额。

第四章 第四节 Python面向对象编程实训

在学习 Python 面向对象编程技术后，以按揭贷款为案例，利用 Python 面向对象技术，对其进行建模，实践面向对象分析与编程。

本关的编程任务是补全 4.4.py 文件中 findPayment、__init__、makePayment 以及 getTotalPaid 四个函数，以实现计算按揭贷款的要求。具体要求如下：

- 本关要求补全上述描述的 4 个函数，并通过研究按揭贷款的规则，来帮助人们选择合适的贷款方式；
- 具体请参见后续测试样例。

本关涉及的代码文件 4.4.py 的代码框架如下：

第四章 第四节 Python面向对象编程实训

```
def findPayment(loan, r, m):
    # 请在此添加代码，补全函数findPayment
    #***** Begin *****#
    #***** End *****#

class Mortgage(object):
    def __init__(self, loan, annRate, months):
        # 请在此添加代码，补全函数__init__
        #***** Begin *****#
        #***** End *****#
        self.legend = None
    def makePayment(self):
        # 请在此添加代码，补全函数makePayment
        #***** Begin *****#
        #***** End *****#
    def getTotalPaid(self):
        # 请在此添加代码，补全函数getTotalPaid
        #***** Begin *****#
        #***** End *****#
    def __str__(self):
        return 'The Mortgage is {self.legend}, Loan is {self.loan}, Months is {self.months}, Rate is {self.rate:.2f}, Monthly
payment is {self.payment:.2f}'.format(self=self)
if __name__ == "__main__":
    print(Mortgage(100000, 6.5, 36))
    print(Mortgage(100000, 6.5, 120))
```

第四章 第四节 Python面向对象编程实训

测试说明

平台会对你编写的代码进行测试：

测试输入：无输入

预期输出：

The Mortgage is None, Loan is 100000, Months is 36, Rate is 0.01, Monthly payment is 3064.90 The Mortgage is None, Loan is 100000, Months is 120, Rate is 0.01, Monthly payment is 1135.48

第四章 第四节 Python面向对象编程实训

第2关：三种贷款方式建模

任务描述

基于上一关实现的 Mortgage 类，利用继承机制，分别补全相关函数，实现用三种贷款方式建模的要求。

相关知识

基于上一关介绍的三种贷款方式：

- 固定利息率，不支付任何点；
- 固定利息率，支付一定的点；
- 可变利息率，低引诱利率，随后高利息率。

以 Mortgage 为父类，定义三个子类 Fixed、FixedWithPts 和 TwoRate，这三个子类都利用基类 Mortgage 的 __init__ 函数进行初始化，然后在 legend 属性上，填上自己的贷款类型描述。首先，TwoRate 类有两个利率，新增了 teaserRate 和 nextRate 两个属性，在 teaserRate 到期后，按 nextRate 利率支付利息。此外，FixedWithPts 在还贷之前，已经支付了一定比例的首付，其每月还贷金额是不同的。FixedWithPts 中的 pts 即为首次支付的比例，例如 20%，则 pts 初始化为 20。TwoRate 的每月还贷金额分为两个时期，请注意这两个时期的利率是不一样的，分别为初始化函数中的 teaserRate 和 r。

编程要求

本关的编程任务是，补全 4.4.2.py 文件中 Begin-End 区间的代码，实现用三种贷款方式建模的要求。具体要求如下：

- 本关要求补全 4.4.2.py 文件中，3 个子类定义的 __init__ 函数和 makePayment 函数，以实现三种贷款方式建模的功能；
- 具体请参见后续测试样例。

本关涉及的代码文件 4.4.2.py 的代码框架如下：

第四章 第四节 Python面向对象编程实训

```
def findPayment(loan, r, m):  
    return loan * ((r * (1 + r) ** m) / ((1 + r) ** m - 1))  
class Mortgage(object):  
    def __init__(self, loan, annRate, months):  
        self.loan = loan  
        self.rate = annRate / 1200.0  
        self.months = months  
        self.paid = [0.0]  
        self.owed = [loan]  
        self.payment = findPayment(loan, self.rate, self.months)  
        self.legend = None  
    def makePayment(self):  
        self.paid.append(self.payment)  
        reduction = self.payment - self.owed[-1] * self.rate  
        self.owed.append(self.owed[-1] - reduction)  
    def getTotalPaid(self):  
        return sum(self.paid)  
    def __str__(self):  
        return str(self.legend)
```


第四章 第四节 Python面向对象编程实训

```
class Fixed(Mortgage):
    def __init__(self, loan, r, months):
        # 请在此添加代码, 补全函数__init__
        #***** Begin *****#
        #***** End *****#
        self.legend = 'Fixed, ' + str(r) + '%, for ' + str(months) + ' months'
class FixedWithPoints(Mortgage):
    def __init__(self, loan, r, months, pts):
        # 请在此添加代码, 补全函数__init__
        #***** Begin *****#
        #***** End *****#
        self.legend = 'Fixed, ' + str(r) + '%, ' + str(pts) + ' points, for ' + str(months) + ' months'
class TwoRate(Mortgage):
    def __init__(self, loan, r, months, teaserRate, teaserMonths):
        # 请在此添加代码, 补全函数__init__
        #***** Begin *****#
        #***** End *****#
        self.legend = str(teaserRate)\
            + '% for ' + str(self.teaserMonths)\
            + ' months, \n then ' + str(r) + '%, for ' + str(months) + ' months'
    def makePayment(self):
        # 请在此添加代码, 补全函数makePayment
        #***** Begin *****#
        #***** End *****#
        Mortgage.makePayment(self)
```

第四章 第四节 Python面向对象编程实训

```
if __name__ == "__main__":  
    print(Fixed(100000, 6.5, 36))  
    print(Fixed(100000, 6.5, 120))  
    print(FixedWithPoints(100000, 6.5, 36, 20))  
    print(FixedWithPoints(100000, 6.5, 120, 20))  
    print(TwoRate(100000, 9.0, 36, 4.8, 12))  
    print(TwoRate(100000, 7.0, 120, 4.8, 36))
```

第四章 第四节 Python面向对象编程实训

测试说明

平台会对你编写的代码进行测试：

测试输入：无输入

预期输出：

Fixed, 6.5%, for 36 months

Fixed, 6.5%, for 120 months

Fixed, 6.5%, 20 points, for 36 months

Fixed, 6.5%, 20 points, for 120 months

4.8% for 12 months,

then 9.0%, for 36 months

4.8% for 36 months,

then 7.0%, for 120 months

第四章 第四节 Python面向对象编程实训

第3关：比较各种贷款的利弊

任务描述

本关利用上一关实现的三个类，比较三种贷款方式，在同等额度贷款、同等时限下，最后还给银行的总金额。

相关知识

请补全 compareMortgages 函数，compareMortgages 函数的各参数说明如下：

amt：贷款总额；

years：贷款年限；

fixedRate：固定利息率，不预付点数贷款方式的利息率；

pts：预付点数；

ptsRate：固定利息率，但预付一定点数贷款方式的利息率；

varRate1：可变利息贷款的引诱利率；

varRate2：可变利息贷款的第二种利息率；

varMonths：引诱利息月份。

编程要求

本关的编程任务是，补全 `step3/7-3.py` 文件中 Begin-End 区间的代码，以实现比较各种贷款利弊的要求。具体要求如下：

- 本关要求补全 compareMortgages 函数，然后分别计算三种贷款方式，最后还给银行的总金额，以此为依据，来对比三种贷款方式的利弊；

- 具体请参见后续测试样例。

本关涉及的代码文件 `7-3.py` 的代码框架如下：

第四章 第四节 Python面向对象编程实训

```
def findPayment(loan, r, m):  
    return loan * ((r * (1 + r) ** m) / ((1 + r) ** m - 1))  
class Mortgage(object):  
    def __init__(self, loan, annRate, months):  
        self.loan = loan  
        self.rate = annRate / 1200.0  
        self.months = months  
        self.paid = [0.0]  
        self.owed = [loan]  
        self.payment = findPayment(loan, self.rate, self.months)  
        self.legend = None  
    def makePayment(self):  
        self.paid.append(self.payment)  
        reduction = self.payment - self.owed[-1] * self.rate  
        self.owed.append(self.owed[-1] - reduction)  
    def getTotalPaid(self):  
        return sum(self.paid)  
    def __str__(self):  
        return str(self.legend)
```

第四章 第四节 Python面向对象编程实训

```
class Fixed(Mortgage):
    def __init__(self, loan, r, months):
        Mortgage.__init__(self, loan, r, months)
        self.legend = 'Fixed, ' + str(r) + '%, for ' + str(months) + ' months'

class FixedWithPoints(Mortgage):
    def __init__(self, loan, r, months, pts):
        Mortgage.__init__(self, loan, r, months)
        self.pts = pts
        self.paid = [loan * (pts / 100.0)]
        self.legend = 'Fixed, ' + str(r) + '%, ' + str(pts) + ' points, for ' + str(months) + ' months'

class TwoRate(Mortgage):
    def __init__(self, loan, r, months, teaserRate, teaserMonths):
        Mortgage.__init__(self, loan, teaserRate, months)
        self.teaserMonths = teaserMonths
        self.teaserRate = teaserRate/1200
        self.nextRate = r / 1200.0
        self.legend = str(teaserRate)\
            + '% for ' + str(self.teaserMonths)\
            + ' months, \n then ' + str(r) + '%, for ' + str(months) + ' months'

    def makePayment(self):
        if len(self.paid) == self.teaserMonths + 1:
            self.rate = self.nextRate
            self.payment = findPayment(self.owed[-1], self.rate, self.months - self.teaserMonths)
        Mortgage.makePayment(self)
```

第四章 第四节 Python面向对象编程实训

```
def compareMortgages(amt, years, fixedRate, pts, ptsRate, varRate1, varRate2, varMonths):
    # 请在此添加代码，补全函数compareMortgages
    #***** Begin *****#
    #***** End *****#
    for m in range(totMonths):
        # 请在此添加代码，补全函数compareMortgages
        #***** Begin *****#
        #***** End *****#
    for m in morts:
        print(m)
        print('Loan ' + str(amt) + ' Total payments = ' + str(int(m.getTotalPaid())))
if __name__ == "__main__":
    compareMortgages(200000, 30, 7, 3.25, 5, 4.5, 9.5, 48)
    print('*'*40)
    compareMortgages(1000000, 30, 7, 20, 5, 4.5, 9.5, 48)
    print('*' * 40)
    compareMortgages(500000, 10, 7, 20, 5, 4.5, 9.5, 48)
```

第四章 第四节 Python面向对象编程实训

测试说明

平台会对你编写的代码进行测试：

测试输入：无输入

预期输出：

第四章 第四节 Python面向对象编程实训

```
Fixed, 7%, for 360 months
Loan 200000 Total payments = 479017
Fixed, 5%, 3.25 points, for 360 months
Loan 200000 Total payments = 393011
4.5% for 48 months,
then 9.5%, for 360 months
Loan 200000 Total payments = 551444
*****

Fixed, 7%, for 360 months
Loan 1000000 Total payments = 2395088
Fixed, 5%, 20 points, for 360 months
Loan 1000000 Total payments = 2132557
4.5% for 48 months,
then 9.5%, for 360 months
Loan 1000000 Total payments = 2757224
*****

Fixed, 7%, for 120 months
Loan 500000 Total payments = 696650
Fixed, 5%, 20 points, for 120 months
Loan 500000 Total payments = 736393
4.5% for 48 months,
then 9.5%, for 120 months
Loan 500000 Total payments = 678254
```